OutOfRole Handbook

The Debugging Table · IBM Z Datathon 2026 · Everything in one place: what we built, the evidence, the event plan and the submission

Contents

The project in one minute 1

The data we test on 2

What we built, part by part 2

Which method we use at each step, and where its code is 3

The baseline: what we compare against 5

Results, backed by data 6

How we got here: the main improvements 8

How we know the results are real 9

Using it in a real company 9

How we use the LLM (Granite) 10

Our proposal: what changed and what to submit 10

What we say openly 12

Event day plan 13

Questions you may have 14

Glossary 15

Where everything is 15

The project in one minute

Some employees misuse access they legitimately have. They copy files to a USB stick before quitting, upload documents to a leak site, or log into colleagues' computers to snoop. Their logins are valid, so simple rules either miss them or bury analysts in alerts.

OutOfRole helps a security team catch these people while they are still active:

- **Watch.** Every day, it compares each employee's last week with their own normal weeks. The 20 most unusual people form that day's review list.
- **Snooping review.** A Netflix-style recommender learns which computers each person normally uses. A periodic review flags people who keep logging into computers outside their profile.

- **Explain.** For each flagged person, code ranks the evidence. IBM Granite picks a label and the supporting facts, and code writes every sentence, so the AI cannot make things up.
- **Advise on access.** The recommender predicts which websites each person will use next, so routine access can be pre-approved, and websites people stopped using become access-review suggestions.

Everything runs on one IBM LinuxONE server, and people appear only as pseudonyms (U-017).

Headline: in a day-by-day replay of 2,000 employees' logs, our 20-name daily list caught **80 of 99 insiders while they were still active**, twice the 40 caught by a standard baseline with the same 20-name list. The baseline is explained in its own section below.

The data we test on

We use the **CERT Insider Threat dataset** from Carnegie Mellon, the standard benchmark for this problem. It is synthetic: generated logs of a fake company, with a separate answer key listing who the insiders are and what they did.

Release	Staff	Insiders	Months of logs
r4.2	1,000	70	about 17
r5.2	2,000	99	about 17

The logs cover logons (when, which computer), USB use, file copies, web browsing and email. The insiders follow four attack patterns:

Attack type	What they do	In
Leakers	Start working after hours, use USB sticks, upload data to wikileaks.org	both
Job-hunters	Browse job sites, then copy data to USB at a higher rate than before, then leave	both
Disgruntled sysadmins	Download a keylogger, plant it on a supervisor's computer, use the stolen login	both
Snoopers	Log into colleagues' computers, search for files, email them home, more and more often	r5.2 only

How we use it. The early months are "training": we learn what normal looks like for each person. Then we replay the later months one day at a time, as if live, and only use the past on each day. The answer key is used only to grade the results, never to build or tune the detector. We fixed every setting on r4.2, then ran r5.2 unchanged, so r5.2 is the fairer test.

What we built, part by part

1. The daily review list (the main detector)

For each person and each day, we take their last 7 days of activity: logons, after-hours logons, USB use, after-hours USB use, file copies, after-hours file copies, web browsing, after-hours browsing, and the number of computers used. We compare each count with **every one of that person's normal weeks** from training. If this week is higher than all of them, that activity counts as unusual. The person's score is the number of unusual activities. Scores build up over a week, so persistent oddness rises and a one-off spike fades. The 20 highest scores form the day's list.

2. The snooping review (the recommender's detection job)

The recommender (an ALS model, the same kind of collaborative filtering Netflix-style systems use) learns which computers each person normally logs into. A login to a computer that is new to the person and that the recommender

scores as unlikely for them is flagged. The review counts the weeks with such logins and discounts people who always move between computers anyway, such as IT staff.

3. The case file (where Granite comes in)

For each flagged person, code ranks what they did: things not seen in their training history first, then the rarest in the company. The top items become an evidence pack, with an ID for each fact (E1, E2, …). Granite replies with only two things: a label from a fixed list ("possible data theft", "possible data leak", "possible snooping", "unusual after-hours activity", "weak evidence") and the IDs of the supporting facts. Code writes every sentence from the evidence pack and always shows the top 3 items. Any other reply is rejected and replaced by a fixed fallback. That is how we guarantee the AI cannot invent a fact.

4. Access advice (the recommender's second job)

The same kind of recommender learns which websites each person uses and predicts which new ones they will use next. Those can be pre-approved, so reviewers see fewer routine requests. Websites a person used in training but not since become suggestions for removing access.

5. New-site rule

Visits to websites no one in the company had used during training are flagged. In CERT, this catches every wikileaks and keylogger-site visit. For a real company, the practical version is to hold **uploads** to never-seen or newly registered sites, not to block browsing (see "Is it practical?").

Which method we use at each step, and where its code is

The build uses two kinds of machine learning plus one language model. All of it is **unsupervised**: nothing learns from the answer key, because a real company never has a list of its insiders. We tried supervised models and they failed on attacks they had not seen (see the second table).

Step	Method (what kind of ML)	What it learns, from what	What it outputs	What it buys us (r4.2 / r5.2)
0. Data tables	**Spark** distributed processing (not ML)	Nothing; it aggregates 15 to 37 GB of raw logs	Three tables per person per day: activity counts, computers used, websites visited	Everything after this runs in about 20 seconds per release
1. Daily review list	**Unsupervised anomaly detection, one model per person:** an empirical percentile test ("is this week above every one of my normal weeks?") on 9 activity counts, with a score that halves every 7 days	Each person's own distribution of weekly activity, learned from their training weeks	A daily score per person; the top 20 form the list	Caught in time: 59 / 80, against 59 / 40 for the baseline. An active insider is on the list 64x / 82x as often as a normal employee

Step	Method (what kind of ML)	What it learns, from what	What it outputs	What it buys us (r4.2 / r5.2)
2. Snooping review	**Collaborative filtering:** implicit-feedback ALS matrix factorization (the Netflix-style recommender), on the person-by-computer usage matrix	Which computers "people like you" log into	For each login, how unexpected it is for that person; weeks with unexpected logins are counted, discounting people who always roam	27 of 30 snoopers in the top 100 (26 without the recommender), r5.2 only
3. Evidence ranking	**Frequency statistics:** novelty (not in this person's history) and rarity (share of staff who used it), no training	How common each computer, website, USB or after-hours action is, from training	The top items for each flagged person, each with an ID	A real malicious action in the top 3 for 64% / 85% of insiders caught in time, against 49% / 51% for rarity alone
4. Case file	**Large language model (IBM Granite), used as a constrained classifier:** it picks an attack-type label from a fixed list and the IDs of the supporting facts	Nothing new; it reads the evidence pack	A label plus fact IDs; code writes every sentence	A readable case file for analysts and auditors that cannot contain invented facts; label accuracy is measured at the event
5. Access advice	**Collaborative filtering again:** ALS matrix factorization on the person-by-website matrix, used as a top-20 recommender	Which websites people with similar habits use	Each person's 20 most likely next websites (pre-approve) and websites they stopped using (review)	Predicts 21% / 19% of next new websites, against 14.5% / 13% for popularity; access requests down 21% / 19%, 28% with monthly retraining
6. New-site rule	**A simple rule** (not ML): never used by anyone in training	The set of websites seen in training	Flags for first-seen sites	All wikileaks and keylogger-site visits flagged; for a real company, flag uploads only

Where the code is for each step:

Step	Code	Notebook
0. Data tables	`code/build_tables.py`	Step 1
Baseline	`outofrole_minimal.py`, `r2_score()` at line 58	Step 3
1. Daily review list	`outofrole_minimal.py`, lines 43 to 94	Steps 10 and 19
2. Snooping review	`outofrole_minimal.py`, lines 97 to 122	Step 13
3. Evidence ranking	`outofrole_minimal.py`, lines 124 to 186	Step 18
4. Case file	`code/case_file.py`	Step 18, Part 3
5. Access advice	`outofrole_minimal.py`, lines 188 to 215	Steps 16 and 17
6. New-site rule	`outofrole_minimal.py`, lines 217 to 221	Step 17

The baseline we beat is also unsupervised and per person. It compares the last 28 days of five activity counts with what the person's own training-period daily rates predict, using a standardized excess: (observed minus expected) divided by the square root of (expected plus 1).

Methods we tried and dropped, all in the study notebook:

Method	Kind	Why we tried it	What happened	Where
Logistic regression, gradient boosting	Supervised classifiers trained on labelled insiders	Usually the strongest when labels exist	Strong on known attacks (up to 60 of 70), but with job-hunters hidden from training they found only 4 to 7 of 30: they memorize attack types	notebook Steps 5 to 7
Isolation Forest, LOF, autoencoder	Unsupervised outlier detectors on daily and weekly behaviour	Standard in published work on this dataset	Weekly LOF was best at slow job-hunter drift (12 to 13 of 30); none beat the per-person percentile on its own	notebook Steps 6 and 8
Fusion of several signals (Stouffer, Fisher, Cauchy)	Statistical combination of signal scores	Combine strengths without labels	55 / 43 insiders in the top 100 reviewed, against 44 / 39 for the baseline; kept as a comparison, not in the build	notebook Steps 11, 12 and 14
Active learning from analyst verdicts (AAD)	Semi-supervised re-weighting after each review round	Learn from analysts without overfitting	57 / 42 against 55 / 43: no real gain	notebook Step 20 (summary)
Item-kNN, user-kNN, Markov chains, richer recommender items	Other recommender and sequence models	Give the recommender more context	Better predictions of normal use, but no gain at spotting insiders over simple popularity	notebook Step 20 (summary)

The baseline: what we compare against

What it is. The standard "is this person acting unusually compared with their own habits?" check that a basic monitoring tool would run. We call it the 28-day personal baseline.

How it works. For each person, take the last 28 days and count five things: USB use, file copies, after-hours logins, after-hours USB use and after-hours web browsing. Compare each count with what that person's own training-period habits predict for 28 days. The bigger the excess, the higher the score. The 20 highest scores form its daily list.

How we measure both. We replay the test months day by day with only past data. Both methods get the same 20-name daily list. An insider counts as **caught in time** if they appear on the list at least once between their first and last malicious action. We also compare against chance (random lists), which catches about 3 insiders.

What we do differently to beat it.

1. "Higher than every normal week" instead of "higher than average": averages are thrown off by people who are naturally bursty.
2. More signals, including the number of computers used, which is how snooping shows up.
3. A score that builds up over a week, so persistent behaviour stands out.

Where each one wins (insiders caught in time, r4.2 / r5.2):

Attack type	Our daily list	Baseline
Leakers	25/30 / 26/29	25/30 / 23/29

Attack type	Our daily list	Baseline
Job-hunters	**30/30 / 27/30**	25/30 / 7/30
Snoopers (r5.2 only)	**25/30**	2/30
Disgruntled sysadmins	4/10 / 2/10	**9/10 / 8/10**
All insiders	**59/70 / 80/99**	59/70 / 40/99

That is why r5.2 shows twice the catches (it has snoopers and harder job-hunters) while r4.2 ties. The baseline is clearly better on the two-day sysadmin sabotage.

Results, backed by data

Level by level

Each level runs on the output of the one above it. Grey bars show the reference (chance or the previous best); teal bars show the level. r4.2 has 70 insiders among 1,000 staff; r5.2 has 99 among 2,000.

All numbers below come from one program, `code/outofrole_minimal.py`, and are listed with their sources in `evidence/Verified_Numbers.md`.

Catching insiders in time

Result	r4.2	r5.2
Insiders on our daily list before their last malicious action	59 of 70	**80 of 99**
Same, 28-day baseline with the same 20-name list	59	40
Same, random score with the same list size	10	11
Average share of each insider's malicious actions that come after we flag them	54%	48%

Do insiders just appear, or appear more often?

Much more often. This matters: if innocent people appeared as often, the list would be useless.

On a given day	r4.2	r5.2
A normal employee is on our list	1.3% of days	0.7% of days
An insider, during their attack, is on our list	82% of days	54% of days
So an active insider is on the list	**64x as often**	**82x as often**
Same comparison for the baseline	39x	47x
Insiders' share of list slots (their share of staff)	40% (7%)	37% (5%)

The honest trade-off: over a whole year, 25% (r4.2) and 15% (r5.2) of normal staff appear on our list at least once, usually briefly (median 9 to 10 days in the whole year). The baseline puts fewer different people on its list but keeps them there about twice as long.

Evidence for the case files (before Granite)

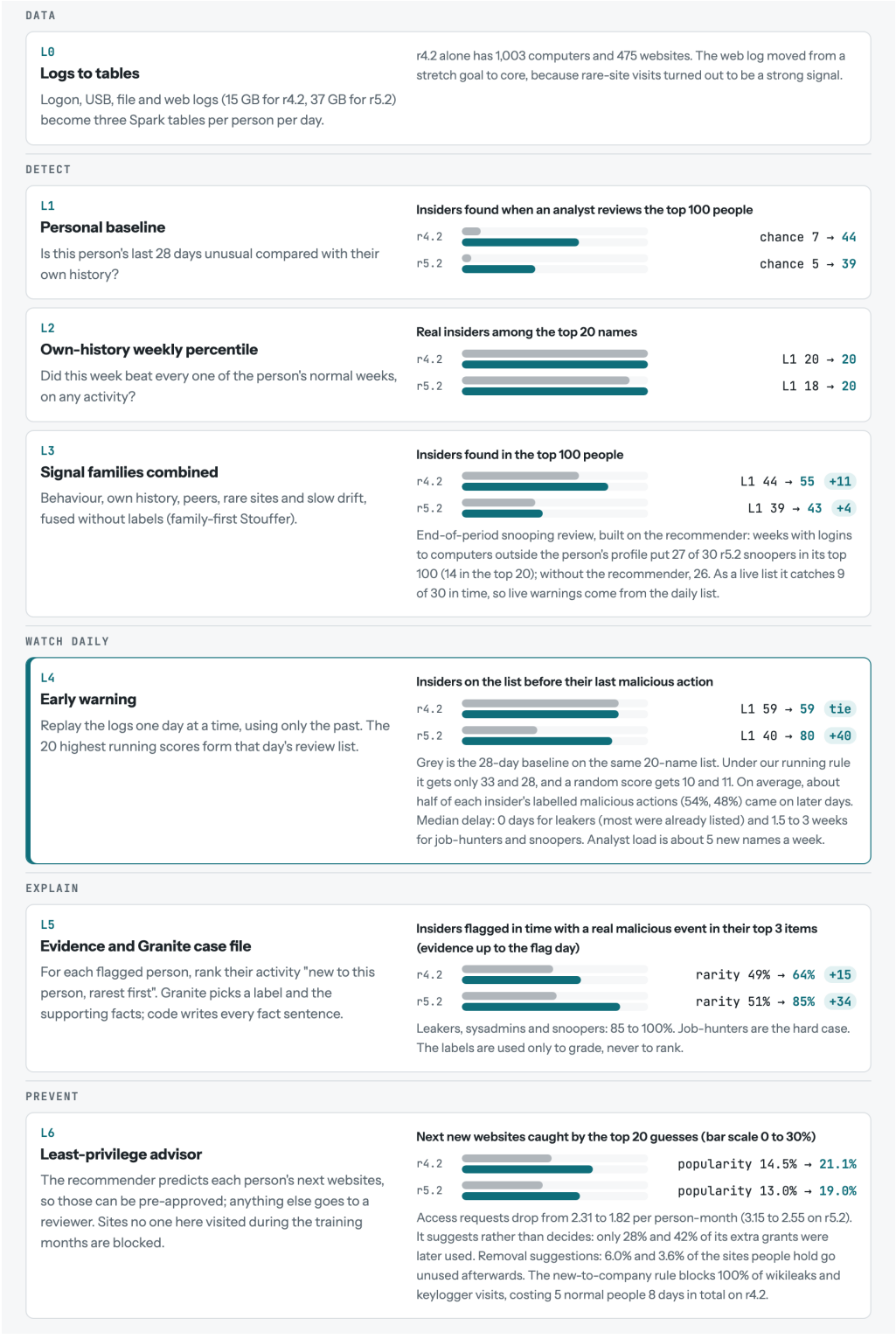


Figure 1: Level by level

Result	r4.2	r5.2
Insiders caught in time whose top 3 evidence items include a real malicious action	64%	85%
Same, ranking by rarity alone	49%	51%
"Strong evidence" at first alert: insiders vs normal staff (no labels used)	29% vs 18%	65% vs 31%

The recommender

Result	r4.2	r5.2
Next new websites caught by its top 20 guesses (popularity in brackets)	21% (14.5%)	19% (13%)
Access requests per person per month: history only, then with the recommender	2.31 to 1.82	3.15 to 2.55
Same, retrained monthly with approvals kept: history only / trained once / retrained monthly	2.24 / 1.76 / 1.61	3.08 / 2.50 / 2.21
Websites used in training but not again (access-review suggestions)	6.0%	3.6%
Snooping review: snoopers in its top 100 (without the recommender)	no snoopers	27 of 30 (26)
Same review as a live 10-name daily list: snoopers caught in time	no snoopers	9 of 30
Wikileaks and keylogger-site visits caught by the new-site rule	100%	100%

How we got here: the main improvements

We tested about 20 ideas in order. Each one either earned its place or was dropped with evidence.

Step	What we tried	Before	After	Verdict
Baseline	28-day personal baseline	(none)	44 / 39 insiders in the top 100 reviewed people	the bar to beat
Trained models	Learn from labelled insiders	39 found	45 to 60 found	dropped: with an attack type hidden from training they found only 4 to 7 of 30 job-hunters
Label-free scoring	No labels at all	trained: 4 to 7	16 of 30 hidden job-hunters	kept
Own-history percentile	"Higher than every normal week"	18 of top 20 real insiders (r5.2)	20 of 20 on both releases	core of the daily list
Combined signals	Fuse several label-free signals	44 / 39 in the top 100	55 / 43	kept as a comparison
Recommender as detector	Flag surprising websites	popularity	no gain on any item type	moved to other jobs
Recommender tuned properly	Settings chosen without labels	popularity 14.5%	21.1% next-site recall	real job: access advice

Step	What we tried	Before	After	Verdict
Learning from analyst verdicts	Re-weight after each review round	55 / 43	57 / 42	dropped: noise
Day-by-day replay	The daily list	baseline 59 / 40 caught in time	59 / 80	the headline
Evidence ranking	"New to you, rarest first"	rarity 49% / 51%	64% / 85%	used for case files
Recommender snooping review	Computers outside the profile	any new computer: 26 of 30	27 of 30	kept as a periodic review

How we know the results are real

- **Strict time split.** Training months come first, then the replay, and on each day only the past is used.
- **Labels only for grading.** No setting was fitted to the answer key.
- **Frozen settings.** Everything was fixed on r4.2, then run unchanged on r5.2.
- **Unseen attacks.** Hiding an attack type from trained models showed they overfit (4 to 7 of 30), which is why the final system uses no labels.
- **Resampling.** Over 5,000 resamples of the insiders, our lead on r5.2 stays between +27 and +52 and is never zero or negative. On r4.2 it is a genuine tie (-6 to +6).
- **Controls.** A random score with the same list size catches 10 and 11. Moving the training cutoff a month later changes nothing.
- **Independent checking.** A separate validator re-ran every script and found no use of future data or answer-key leakage. One mislabelled number was corrected.
- **Outside review.** Codex reviewed the work in 11 rounds; its final verdict on simple, impressive and connected to the proposal was "yes" on all three.
- **Reproducible.** One 253-line program rebuilds every headline number in about 20 seconds per release.

What remains a risk: a few choices were made after seeing results (the daily-list method, the snooping review, the training cutoff date). The fix is a dataset we have never looked at, which is why we are adding the LANL dataset (see the FAQ).

Using it in a real company

Is it practical?

Part	Practical?	How to present it
Daily list and case files	Yes. This is how commercial insider-risk tools work (Microsoft Purview Insider Risk, Sentinel UEBA).	A triage aid for analysts, not an accusation tool. In a real company insiders are far rarer than CERT's 5 to 7%, so most names will be innocent.
Snooping review	Yes, as a monthly report.	"Who keeps logging into computers outside their role?"
Case files with Granite	Yes. The constrained design suits compliance and audits.	Granite labels and picks facts; code writes the text.
Access advice	Yes, but for applications, shared drives and groups. Companies do not manage website access per person.	An access-review assistant. CERT has no permission lists, so websites stand in.
New-site rule	Not as built: blocking every never-seen site is excessive. It only works because CERT has about 470 websites in total.	Hold or flag uploads to never-seen or newly registered sites.

Keeping it relevant as people and roles change

- **CERT never changes anyone's role** (0 role changes), though 14 to 30 people change department and 126 to 213 change team. So we tested the general fix: let each person's "normal" roll forward, using their last 26 weeks and skipping the most recent 4 so a slow attack cannot become the new normal.
- **Detection held:** 59 of 70 and 84 of 99, against 59 and 80 with frozen normal weeks. The cost is that more innocent people appear on the list at some point (274 vs 230, and 374 vs 291).
- **Retrain the recommender monthly.** It trains in seconds. Monthly retraining cuts access requests 28% below own history alone, against about 21% when trained once.
- **For a real deployment:**
  - update each person's normal weekly
  - when HR records a role or department change, compare the person with their new team for a few weeks and note the change in the case file
  - keep confirmed incidents out of anyone's "normal"
  - retrain the recommender monthly

How we use the LLM (Granite)

Now: Granite labels each case and chooses the supporting facts from the evidence pack; code writes every sentence and rejects anything else. A self-test sends 12 bad replies (made-up names and quantities, unknown facts, broken JSON) and all of them fall back safely.

Planned for the event, in order of value:

1. **Measure its labels.** Compare Granite's attack-type label with the answer key; chance is 1 in 4 on r5.2. This turns "AI explains" into a measured number.
2. **Daily briefing.** Granite summarizes the day's list (who is new, who escalated, which pattern), using the same labels-and-IDs approach.
3. **"Ask your logs."** An analyst types a question in plain English; Granite picks one of a fixed set of queries and fills in the details; code runs it. A strong live demo on LinuxONE.
4. **Access-review notes.** Granite drafts the reason behind each removal or pre-approval suggestion from the facts.

Our proposal: what changed and what to submit

We promised	We deliver
"A recommender that catches insider snooping" (our title)	The recommender's snooping review finds 27 of 30 snoopers; live, the daily list catches 25 of 30 in time
"Learns which files and machines each employee normally uses"	Computers and websites. File identities were not modelled: 100% (r4.2) and 94% (r5.2) of CERT file names belong to one person, which leaves nothing to learn
"Access it would never suggest is flagged"	Snooping review for computers; pre-approval and review for websites
"An AI on the same server explains why"	Granite case files on LinuxONE
Analysts, compliance officers, access admins	Daily list; dated, cited case files; access suggestions
Data never leaves the platform	One server, pseudonyms only
Spark, ALS on early months, later months replayed live	Exactly that

What changed: each person's own history catches insiders better than the recommender does, so it drives the broad daily list. The recommender does two jobs where it measurably helps: the snooping review and access advice.

Did the recommender work?

As the detector, no. In two supporting jobs, yes.

Job	Result (r4.2 / r5.2)	Verdict
Spotting insiders across all attack types	Never beat simple popularity, on any item type	Did not work
Live snooping alarm	9 of 30 snoopers caught in time (r5.2)	Did not work
End-of-period snooping review	27 of 30 snoopers in its top 100, 26 without it	Works, small edge
Predicting the access each person will need	21% / 19% of next new websites, vs 14.5% / 13% for popularity	Works
Fewer access requests	21% / 19% fewer; 28% with monthly retraining	Works

What actually catches insiders is each person's own normal week: 80 of 99 caught while still active on r5.2, against 40 for a standard baseline.

Final submission text, field by field

Each field fits the original form's limits.

Project: OutOfRole: early warning for insider misuse, explained by Granite on LinuxONE.

Problem statement (173 of 200 characters): Insiders misuse valid access to leak, steal or snoop. Rules miss them or flood analysts. Teams need early, explained warnings without moving sensitive data off the platform.

Proposed solution (179 of 200): OutOfRole learns each employee's normal week and lists the 20 who drift most each day. Granite on LinuxONE labels each case from checked evidence; a recommender advises on access.

Audience (785 of 1,024): (1) Security analysts: a 20-name daily review list instead of thousands of rule hits. In a day-by-day replay it caught 80 of 99 insiders while still active, twice a standard baseline. (2) Privacy and compliance officers: dated case files that cite their evidence; the AI picks a label and the facts, code writes every sentence, so nothing is invented. (3) Identity and access admins: a Netflix-style recommender predicts the access each person will need (1.45 times better than popularity), cutting access requests by about a fifth, and lists access people stopped using for review. (4) Patients and account holders: monitoring and AI stay on the platform; people appear only as pseudonyms. (5) Organizations running core records on IBM Z: everything runs next to the data on LinuxONE.

Data and method (685 of 1,024): (1) CMU SEI CERT Insider Threat r4.2 and r5.2: synthetic logons, USB, file, web and email logs for 1,000 and 2,000 employees, with an answer key of 70 and 99 insiders used only for grading. (2) Spark builds per-person daily tables; each person's normal weeks are learned from the early months, and the later months are replayed day by day as a live stream. (3) An ALS recommender learns who uses which computers and websites, for snooping reviews and access advice. (4) IBM Granite labels each case from a ranked evidence pack with a fail-closed check. (5) Settings frozen on r4.2, tested unchanged on r5.2; LANL's real authentication logs with red-team labels as an independent check.

The 30-second pitch

"Malicious insiders were the costliest attack vector in IBM's 2025 breach report, and they hide behind valid logins. OutOfRole learns each employee's normal week and gives analysts a 20-name daily list. In a day-by-day replay of 2,000 employees, it caught 80 of 99 insiders while they were still active, twice a standard baseline. For every alert, IBM Granite on the same LinuxONE server labels the case from checked evidence, and code writes the text, so it cannot invent facts. A Netflix-style recommender keeps access lean, cutting access requests by about a fifth. No data leaves the platform."

Why this can win

Judging angle	What we show
Real value	A daily list analysts can work through, explained case files for compliance, leaner access for admins

Judging angle	What we show
Novelty	We measure catching insiders while they are still active, which much published work on this dataset does not report; the AI cannot invent facts; the recommender's job is measured in fewer access requests
Technical depth	Per-person anomaly detection, ALS recommender, a constrained LLM, a day-by-day replay with no look-ahead, a resampling check (+27 to +52 insiders on r5.2)
LinuxONE and IBM tech	Spark, Granite and the dashboard all on one LinuxONE server; pseudonyms only
Credibility	Settings frozen before the test release, an independent validator, 11 outside review rounds, one 253-line program that reproduces every number

How to talk about the change

Say it plainly and early, as a strength: "We tested our own idea honestly. The recommender turned out to be a great predictor of what people need, but a weak insider detector, so we used it where it wins (access advice and snooping reviews) and built detection on each person's normal week, which doubled the baseline."

Judges reward a team that tests its own assumptions. They punish one that hides a weak result.

Three things that would strengthen it at the event

1. **Measure Granite's labels** against the answer key (chance is 1 in 4 on r5.2), so "the AI explains" becomes a number.
2. **Run on LANL's real logs** (already downloaded: 1.05 billion login events, 749 red-team events) to show the approach is not just tuned to synthetic data.
3. **Combine with the baseline for sysadmins:** the baseline catches 8 to 9 of 10 sabotage cases where our list catches 2 to 4. A combined list is a likely quick win; it needs testing.

What we say openly

- CERT is synthetic data; these are replay results, not a live deployment.
- The daily-list method was chosen after seeing results. The method we had named in advance also beat the baseline (54 vs 28 of 99).
- Two-day sysadmin sabotage is mostly missed; the baseline does better there.
- The snooping review was designed after looking at r5.2; as a live list it catches only 9 of 30 snoopers in time.
- Most leakers were already doing suspicious USB copying before the answer key's start date, so "caught the same day" partly means "already on the list".
- Granite and the LinuxONE run are built at the event; the code they plug into is tested here.

Say this, not that

- Say "flags insiders while they are still active", not "real-time production system". It is a replay of past logs.
- Say "the recommender predicts legitimate access", not "the recommender detects insiders". We tested that: as a detector it is no better than popularity.
- Say openly that two-day sysadmin sabotage is mostly missed, and that CERT is synthetic data.
- Say "suggests which access to pre-approve", not "trims access to what people need". Only 28% (r4.2) and 42% (r5.2) of the recommender's extra grants were later used.
- Say plainly that the approach changed after testing. The abstract promised recommender-based detection; our tests showed that own-history baselines detect better, so the recommender moved to access advice.
- Compare against the strongest baseline. On the same 20-name list, the 28-day baseline flags 59 of 70 (a tie on r4.2) and 40 of 99 (half of ours on r5.2).
- The early-warning method was picked after seeing results. The full detector we had named in advance also beat the baseline: 54 vs 28 of 99 r5.2 insiders flagged in time.

Event day plan

Who builds what, hour by hour

Fill in names. Hours count from the start of our build.

Role	Hours 0 to 6	Hours 6 to 12	Hours 12 to 18	Hours 18 to 24
R1 Data and platform (_____)	VM: Java 17, PySpark (no pyarrow on s390x), Docker; download r4.2. At hour 2, hand a one-month sample to R2 and R4. Spark builds 3 tables: daily counts, person-computer-day, person-site-day	r5.2 if disk allows; run the whole pipeline end to end	Docker container, GitHub repo, README	Support the demo; check the repo runs from scratch
R2 Detection (_____)	Port section 1 of `outofrole_minimal.py` (the daily list) and run it on the sample	Run on r4.2 with the fixed settings below	r5.2 unchanged; results table vs baseline and chance, per attack type, analyst load	Export daily lists and per-person timelines for the dashboard
R3 Granite (_____)	Both IBM labs (Granite via llama.cpp; the model file must be big-endian)	Port `case_items` from `outofrole_minimal.py` and all of `case_file.py` (evidence pack with IDs, Granite prompt that returns only a label and fact IDs, reply check, fallback); pseudonyms	Pre-generate case files for the top 20; keep one live run	Case-file panel in the dashboard
R4 Recommender and demo (_____)	Dashboard skeleton: daily list, person page, case-file panel	Port sections 3 to 6 of `outofrole_minimal.py` ALS on person-site data, top-20 pre-approval, removal suggestions, new-to-company block, and the recommender snooping review (end of period)	Wire everything into the dashboard	Video with on-screen proof: `uname -m` (s390x), `docker ps`, the server prompt

Fixed detector settings (decide nothing new):

- A feature is unusual when the last 7 days beat every one of that person's training weeks.
- The running score halves every 7 days.
- The list holds 20 names a day.

Recommender settings: rank 64, alpha 0.5, regularization 1.0 on log(1 + active days), using the numpy version in `recommender_als_simple.py`. Spark's ALS scales regularization differently, so if you use it, re-tune on the last training month and pick by how many next-month new sites it catches.

Cut rules

- **Hour 6:** if the Spark tables aren't ready, run r4.2 with logon, USB and file data only; add web later.
- **Hour 12:** freeze the detector settings. No tuning after this.
- **Hour 14:** if the recommender is behind, ship the new-to-company block plus each person's own history as the allow-list. Cite the recommender's local result instead.
- **Hour 18:** if Granite is slow or unreliable, show the pre-generated case files plus one live run. If it fails entirely, show the checked evidence pack itself.
- **Hour 20:** feature freeze. Only the video, README and slides from here.

Demo (3 minutes)

1. The replay plays. A leaker joins the daily list on the day of the first wikileaks upload.
2. Click the person. The case file appears: Granite's label, the top-ranked evidence written by code, and the facts Granite cited marked. Then show a bad reply being rejected and replaced by the fallback.
3. Access panel: 20 pre-approved sites. A request for wikileaks.org is blocked as new to the company.

The code to port

In `code/`, in the order they run:

File	What it does	Who ports it
`build_tables.py`	Spark turns the raw logs into the three tables (daily counts, person-computer-day, person-site-day)	R1
`outofrole_minimal.py`	The whole system, printing every headline number: daily list (section 1, lines 43 to 94), snooping review (1b, lines 97 to 122), evidence ranking (2, lines 124 to 186), recommender and access advice (3 and 4, lines 188 to 215), new-site rule (5, lines 217 to 221)	R2: sections 1 and 2; R4: 1b, 3 to 5
`case_file.py`	Evidence pack, Granite prompt, reply check and fallback; run it to see the self-test	R3
`checks/` `followup_checks.py`	Resampling, rolling baseline, role changes, monthly retraining	optional
`experiments/`	The original study scripts behind the evidence	reference only

- Full study with code: `evidence/Study_Notebook.ipynb`
- Data download steps: `setup/Data_Download_Guide.pdf`

Questions you may have

Which parts are machine learning? Two parts learn from data without labels: the per-person "normal weeks" model behind the daily list, and the ALS recommender behind the snooping review and access advice. Granite is a large language model used as a constrained labeller. Spark, the evidence statistics and the new-site rule are data processing and simple statistics. See "Which method we use at each step".

Why no supervised model, when they score higher in the study? They need a list of known insiders, which a real company does not have, and they memorize the attack types they were trained on. With job-hunters hidden from training, they found only 4 to 7 of 30; our label-free approach found 16.

What exactly is "in time"? The insider appears on the daily list at least once between their first and last malicious action, so there is still something to stop.

If innocent people appear on the list too, did we achieve anything? Yes. On a given day, an insider in the middle of an attack is 64 to 82 times as likely to be on the list as a normal employee. Insiders hold 37 to 40% of list slots while being 5 to 7% of staff; a random list would give them 5 to 7%.

Why not just use the baseline? On r5.2 it catches 40 insiders in time against our 80, and misses almost all snoopers (2 of 30) and most job-hunters (7 of 30). It is better on sysadmins, which suggests combining the two lists; we have not tested that yet.

Why did the recommender stop being the main detector? We tested it on websites, computers, website-by-hour, email recipients and browsing order. It never beat simple popularity at spotting insiders, because most attacks either use sites nobody visits (popularity already catches those) or sites that look normal for the person. It is genuinely better at predicting what people will use next, so it moved to access advice and the snooping review.

Can Granite invent facts in a case file? No. Granite returns only a label from a fixed list and fact IDs. Code writes every sentence from the evidence pack. Any other reply is rejected and replaced by a fixed fallback; the self-test proves this with 12 bad replies.

What if Granite does not work at the event? The fallback case file is fully deterministic and still shows the ranked evidence. The demo works without Granite; Granite adds the label.

Did we overfit? We guarded against it at every step (see "How we know the results are real"). The remaining risk is choices made after seeing results; the LANL test addresses it.

What is the LANL dataset, and why add it? Los Alamos National Laboratory published 58 days of real (not synthetic) login records from its network: 12,425 users, about 1 billion login events, plus a list of 749 red-team attack logins. Using it tests our ideas on data we have never seen. Its attacks are outside intruders moving between computers with stolen accounts, so the snooping review (logins outside a person's normal computers) is the closest fit. We requested the data and are downloading it now.

Do we need to retrain when roles change? Yes, cheaply: roll each person's normal forward weekly, treat a role change as a reason to compare with the new team for a few weeks, and retrain the recommender monthly. Our tests show detection holds with a rolling normal and monthly retraining cuts more requests.

Is it impressive? On r5.2, yes: twice the baseline, robust to resampling, with 54 to 48% of each insider's malicious actions still ahead when we flag them. On r4.2 it ties the baseline. The rigor is our main differentiator: independent validation, 11 review rounds, and one short program that reproduces everything.

Why 20 names a day? A small team can review 20 people a day. It also makes the comparison fair, since every method gets the same list size.

How much code is the build? Two files: 253 lines for scoring and 85 lines for case files, on top of the three tables Spark builds. About 20 seconds per release on a laptop.

What runs on LinuxONE? Everything: Spark builds the tables, Python scores and ranks, Granite labels the case files, and the dashboard shows the list. Nothing leaves the server.

What should I say to judges in one sentence? "In a day-by-day replay of 2,000 employees' logs, OutOfRole put 80 of 99 insiders on a 20-name daily review list while their attacks were under way, twice a standard baseline; Granite labels each case from checked evidence; and a recommender handles snooping reviews and access advice."

Glossary

- **Insider:** an employee who misuses legitimate access.
- **Baseline:** the simple standard method we must beat (here, the 28-day personal check).
- **Caught in time:** listed before the insider's last malicious action.
- **Recommender / ALS:** a model that learns "people like you use these things", as in Netflix recommendations.
- **Popularity:** the simplest recommender: guess what most people use.
- **Evidence pack:** the top-ranked facts for one person, each with an ID.
- **Fail closed:** when a check fails, show the safe fallback instead of the risky output.
- **Synthetic data:** generated data that imitates real logs; it has an answer key.
- **Overfitting:** a method that looks good on the data it was tuned on but fails on new data.

Where everything is

This handbook is the central document; everything else supports it.

Folder or file	What is in it	When to use it
`OutOfRole_Handbook.pdf`	This document (also as one online page)	Always start here
`code/`	`build_tables.py` (Spark tables), `outofrole_minimal.py` (the whole system), `case_file.py` (Granite step), `checks/`, `experiments/`	To run or port the code
`evidence/Verified_Numbers.md`	Every number with the script that produced it	Before quoting any number
`evidence/Study_Notebook.ipynb`	The full study with code, outputs and explanations	To see how a result was found
`evidence/Research_Sources.md`	Published work, industry practice, real cases	To cite a source
`evidence/Proposal_As_Submitted_Oct15.md`	The abstract we submitted	To compare with the final wording
`evidence/Reviews/`	All outside review rounds	To show the work was checked
`setup/`	`LinuxONE_Setup_Guide.pdf`, `Data_Download_Guide.pdf`, `IBM_Lab_Materials/`	Before and during the event